

计算机视觉 课程实验报告

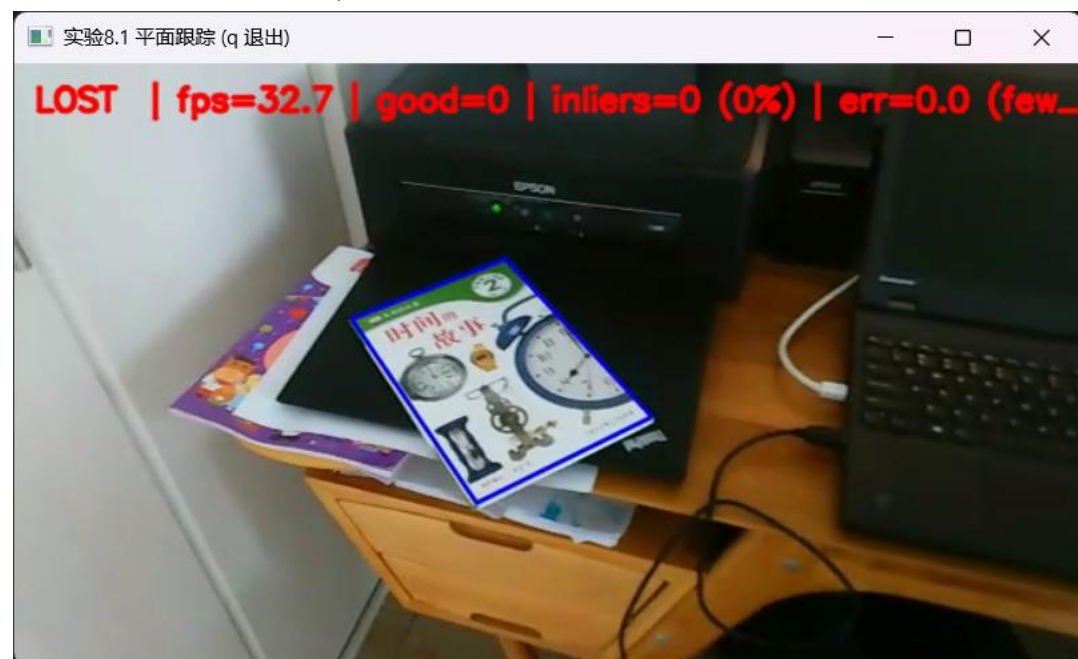
学号：202300130183	姓名：宋浩宇	
实验题目：平面跟踪		
实验过程中遇到和解决的问题： （记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。） 问题 1：首次实现跟踪时，视频中蓝色线框经常出现“乱飞”，在物体被遮挡、运动较快或背景纹理复杂时，线框会跳到错误位置甚至铺满半屏。 解决 1：分析后认为主要原因是特征误匹配和错误单应性被帧间平滑和丢帧保持机制放大。改进措施包括：采用模板与当前帧的双向 <code>ratotest</code> 互匹配，减少单向匹配带来的野点；用 RANSAC 估计单应性后仅用内点重新求精，并检查内点数量和重投影误差；对投影得到的四边形做凸性、面积、长宽比和是否越界等几何校验；与上一帧角点比较，跳动过大则丢弃该帧结果；将平滑系数调低、丢帧保持时间缩短为最多 2 帧，宁可短暂不画框也不显示明显错误的框。默认检测器改为 AKAZE 以提高稳定性，关键筛选逻辑集中在 <code>e8_common.py</code> 的 <code>mutual_ratio_matches</code>、<code>validate_quad</code> 和 <code>PlanarTracker.track</code> 中。 问题 2：加强校验后跟踪更稳，但画面上线框出现时间变少，部分帧显示 LOST 及拒绝原因（如 <code>few_inliers</code>、<code>jump_too_large</code>），容易误以为程序失效。 解决 2：这是“稳定性”和“覆盖率”之间的取舍：旧版约八成以上帧都画框，但其中含不少错误框；新版只在匹配和内点比例、几何与时序条件同时满足时才绘制，默认视频上稳定跟踪帧比例约降至四成多，但乱飞现象明显减轻。通过命令行参数可在二者间调节，例如用 <code>--ratio 0.75</code>、<code>--smooth 0.65</code> 适当放宽以多画框，或用 <code>--ratio 0.65</code>、<code>--smooth 0.45</code> 进一步收紧；预览窗口左上角会显示 <code>good</code> 匹配数、内点比例和重投影误差及拒绝原因，便于定位是匹配不足还是帧间跳变过大导致未通过校验。 问题 3：需要更换自己的模板图和视频进行测试，且不同素材效果差异大，不清楚应如何选图和调参。 解决 3：主程序已支持 <code>--template</code> 和 <code>--video</code> 指定任意路径，不写参数时仍使用 <code>data</code> 下默认素材。实践表明模板宜为平面物体正面、纹理丰富、少大面积纯色，视频中的物体应与模板一致且尽量完整入画。自定义素材跟踪不稳时优先使用 <code>--detectorakaze</code>；追求速度可试 <code>orb</code>。		

实验结果分析：

本实验基于 ORB/AKAZE 特征匹配和 RANSAC 单应性估计，实现了对平面模板的实时跟踪，并在视频帧上用蓝色线框标出物体边沿。在课件提供的默认视频（640 × 360，约 40fps）上，改进前 ORB 方案虽可达约 43 帧/秒且多数帧都有线框，但常出现蓝线乱飞；加入双向互匹配、内点与重投影检验及几何、时序校验后，改用 AKAZE 平均约 38~40 帧/秒，仍满足实时要求，有效跟踪帧约四成到五成，线框能较稳定贴合书本封面，快速运动或遮挡时会短暂丢失而不再明显错位。对比可见，ORB 更快但更易中断，AKAZE 更稳，更适合本实验。自定义素材时，模板清晰、纹理丰富且与视频一致则效果更好。总体而言，实验达到了平面跟踪与实时显示的基本要求，并说明跟踪质量取决于匹配可靠性与单应性结果校验，而非仅看处理帧率。

具体结果如下：

在原本的视频和图片中，得到的结果是：



在我自己录得一部分素材中，得到的结果是：

